

Highlights

MiniEval: Automated Detection of Compliance Violations and Quantitative Privacy Risk Assessment in MiniApps

- Research highlight 1
- Research highlight 2

MiniEval: Automated Detection of Compliance Violations and Quantitative Privacy Risk Assessment in MiniApps

, , , , ,

Abstract

The proliferation of MiniApps has introduced significant and multifaceted privacy risks into the mobile ecosystem. However, two more subtle yet critical problems have often been overlooked: the practice of restricting functionality when a non-essential data request is denied, and the declaration of vague or irrational purposes for data collection within privacy policies. To achieve these, we introduce two concepts: *Indirect Violation* and *Purpose Rationality*. The former, Indirect Violation, identifies instances of coercive functional restrictions. The latter, Purpose Rationality, evaluates the transparency and necessity of declared data collection purposes. This paper presents MiniEval, a framework that integrates these concepts to perform an automated and, ultimately, a quantitative risk assessment. Operationally, MiniEval integrates hybrid analysis with a large language models (LLM) to detect violations and derives a quantitative formula to generate a holistic risk score. Our evaluation on a ground truth set of 350 real-world WeChat MiniApps demonstrates MiniEval’s high efficacy. It achieves a 90.5% macro-averaged F1-score for Purpose Rationality Classification and an 81.5% F1-score for Indirect Violations. Building on this demonstrated efficacy, our large-scale analysis of 15,917 MiniApps leverages this framework to systematically classify applications into risk tiers, establishing that severe privacy risks are both widespread and concentrated in specific application categories.

Keywords: MiniApps, Data Minimization, Privacy Risk Assessments

1. Introduction

MiniApps, a transformative mobile computing paradigm, have rapidly emerged in recent years. MiniApps, operate within large host platforms such as WeChat, Alipay, and TikTok. This architecture provides users with a unified experience for a wide range of activities, from shopping and medical consultations to social games and content sharing, without requiring separate installations. However, integration of MiniApps into host platforms raises significant privacy concerns, including overuse of sensitive user data and lack of transparency in data collection, fueling discussions on privacy risk.

With the growing global attention to privacy protection, regulations such as the General Data Protection Regulation (GDPR) [1], the California Consumer Privacy Act (CCPA) [2], the Personal Information Protection Law (PIPL) [3], and the Canadian Consumer Privacy Protection Act (CCPA) [4] have been successively enacted. Data protection regulations commonly contain three core data protection principles: *purpose limitation*, *data minimization*, and *transparency*. Purpose limitation mandates that data collection must serve a specific and legitimate purpose. Building directly upon this, data minimization restricts data collection solely to what is strictly necessary to achieve that predefined purpose. Finally, the principle of transparency requires that purpose declarations be both clear and easily understandable to the user. In parallel, MiniApp platforms such as WeChat have introduced privacy policies and stricter data review mechanisms [5]. However, in the absence of effective enforcement and regulatory oversight, these laws and policies remain insufficient to protect users from substantial privacy risks, highlighting the urgent need for stronger privacy protection tools.

Research on MiniApp privacy has already produced a variety of tools and frameworks to address privacy concerns. For example, SPOChecker [6] detects over-collection of privacy data in MiniApps, MiniScope [7] constructs dependency graphs to examine the consistency between declared and actual practices, and Shenao Wang et al. [8] employ LLMs to identify data minimization violations from usage scenarios. While valuable, the scope of these existing analyses focus on the consistency between an MiniApp’s behavior and its policy declarations. They overlooks another fundamental issue: the intrinsic validity of the declarations themselves. A purpose declaration that lacks clarity or necessity fundamentally contravenes the data protection principles. This motivates our notion of Purpose Rationality, which assesses whether a declared purpose is rational by examining its clarity, necessity,

and reasonableness. At the same time, existing work has largely overlooked a different but equally important form of misuse: Indirect Violations, where non-essential personal data is not directly over-collected but instead covertly imposed by tying it to functional access, thereby forcing users to disclose unnecessary data.

Our work: Building on these insights, we propose a new perspective that operationalizes data protection principles across both declarative and behavioral dimensions, through the concepts of Purpose Rationality and Indirect Violation. To realize this, we design and implement MiniEval, a systematic framework for automated privacy auditing. First, MiniEval employs an LLM-based reasoning module to classify Purpose Rationality in policy declarations. Second, MiniEval integrates hybrid analysis and a LLM to detect indirect and direct violations detailed in Section 3.1. Furthermore, MiniEval introduces a quantitative risk assessment model, enabling a comprehensive quantitative assessment of privacy risks in the MiniApp ecosystem.

Contributions. This paper makes the following contributions:

- We introduce two complementary concepts: Indirect Violation, which capture covert behavioral misuse, and Purpose Rationality, which evaluates the necessity and clarity of declared purposes.
- We design and implement MiniEval, a systematic framework that combines control-aware privacy flow analysis with LLM-based reasoning to detect both direct and indirect violations of principles. MiniEval further incorporates a classification standard for privacy information sensitivity and produces quantitative risk scores.
- We conduct an empirical study on real-world MiniApps, demonstrating the effectiveness of our framework in identifying violations, classifying purposes, and quantifying privacy risks across the MiniApp ecosystem.

2. Background

In this section, we provide an overview of the MiniApp architecture, the types of user privacy information, and privacy policies.

2.1. The Basic Architecture of a MiniApp

- **Source Code Composition:** The source code of a MiniApp is distributed in a structured format. For example, WeChat MiniApps

include several key file types: *.json* files for configuration metadata, *.js* files for business logic and API access, *.wxml* files for defining the user interface (UI), and *.wxss* files for styling the interface.

- **Architecture and Interaction Mechanism:** As shown in Figure 1, the MiniApp runtime architecture consists of two main components: the view layer and the logic layer. The view layer is responsible for rendering the UI, while the logic layer handles data manipulation, event responses, and API requests. These two layers communicate through the JSBridge mechanism, which underpins the framework’s event-driven interaction model. This bidirectional communication channel operates as follows. When the logic layer updates data, the *setData()* method is called, and the JSBridge propagates this change to the view layer for UI updates. Conversely, User interactions in the view layer, such as inputs, are encapsulated as event objects and delivered to the logic layer via JSBridge. This cyclical exchange ensures real-time synchronization between the view layer and the logic layer.
- **Routing and Sub-package Mechanisms:** MiniApps adopt a page-based routing architecture. Each page is encapsulated with specific logic and presentation, and the pages are connected through a routing mechanism to support navigation. Additionally, MiniApps implement a sub-package loading strategy [9], which improves performance and reduces initial load time. Only the main package is loaded during MiniApps startup; Sub-packages corresponding to individual pages are dynamically loaded upon access. This design supports lightweight execution and enhances the overall user experience.

2.2. Privacy Information and Privacy Policies in MiniApps

2.2.1. Privacy Information in MiniApps

In MiniApps, the collection of private data primarily occurs through two channels: API invocations and user input. Among these, Zhang et al. [6] classified the privacy-related APIs of MiniApps into 20 categories based on the types of user information they collect. These APIs cover a wide range of privacy information, from basic personal details to more sensitive data. For instance, the `getLocation` API retrieves a user’s precise geographical location. Notably, such AP Is adopt an asynchronous execution model, where

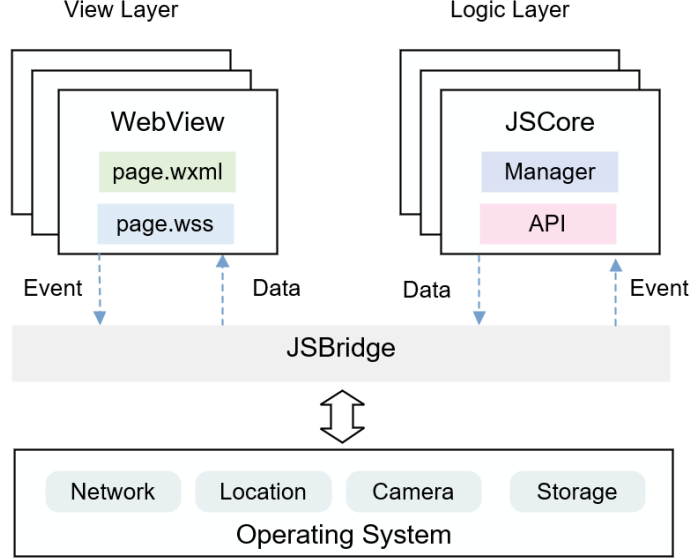


Figure 1: The architecture of MiniApps.

callback functions like `success` and `fail` handle the outcomes of user authorization—grant or denial, respectively. This mechanism introduces implicit control dependencies, as the user’s authorization decision directly determines the subsequent execution path of the program. In addition to API invocations, user input during MiniApp usage may also contain privacy information. For instance, when users fill out forms or utilize search boxes, they may input various types of personal information, including names, addresses, phone numbers, and so on. In the study [6], 17 categories of privacy-related user input were identified. Furthermore, we partition all privacy information items based on their sensitivity, as presented in our *privacy sensitivity classification table*. This classification highlights the varying degrees of harm that different types of information exposure may cause. Both categories of privacy-related user input and the classification table are available in our online documentation [10].

2.2.2. Privacy Policies in MiniApps

Developers of MiniApps are required to fill out and comply with relevant privacy policies and user agreements when handling user information. Before users begin using the service, developers must explicitly present their privacy

policies and obtain informed consent. These policies should clearly specify the types of information to be collected and the purpose. However, in practice, many privacy policies fail to adequately disclose the data being collected or clearly define how it will be used. In some cases, developers may even intentionally conceal relevant details or provide information that is inconsistent with their actual data practices. As a result, significant discrepancies often exist between documented policies and actual program behavior.

3. Motivating Case

3.1. Motivating Case: Indirect Violations of principles in MiniApps

Existing work on violation detection of data protection principles in MiniApps primarily involves contrasting declared privacy policies with observed collection behaviours. This uncovers overt violations, but often misses covert violations. We therefore contrast *Direct Violations* with *Indirect Violations*.

Direct Violations. We systematically classify direct violations into the following three categories:

- *Omitted Disclosure:* Collection of privacy information that is not in the privacy policy, a fundamental breach of the transparency principle.
- *Redundant Disclosure:* This violation occurs when a privacy policy declares the collection of privacy information which the application never actually collects. This practice violates the transparency principle by providing inaccurate information.
- *Scenario Inconsistency:* This violation occurs when the context of data collection deviates from the declared scenario. For example, an MiniApp might state that location is used only for check-ins, yet request it on the user’s profile page. This discrepancy breaches the data minimization principle by tying the data collection to an undeclared scenario. It also undermines the transparency principle by subverting the user’s reasonable expectations, thereby impairing the basis for informed consent.

Indirect Violations. Unlike Scenario Inconsistency, Indirect Violation represents a more subtle form of non-compliance. In an Indirect Violation, a MiniApp restricts access to certain functionalities, making their availability

contingent upon the user’s disclosure of non-essential data. This enforcement is not an incidental outcome but is intentionally implemented within the program’s control logic. Specifically, this logic makes the execution of critical operations, such as navigation and UI rendering, directly contingent upon the user’s disclosure of private information. This coercive pattern undermines data protection principles. By obscuring the mandatory link between functionality and non-essential data, this practice subverts the transparency principle and invalidates user consent. Furthermore, it subverts the data minimization principle by making such data a prerequisite for functionality.

Case 1 — Functional restriction tied to privacy-related API. As shown in Figure 2a, a shopping MiniApp restricts access to the product information page and member benefits information unless disclosure of location. In this case, the permission request and the functional restriction are separated deliberately. The two are linked by the `hasLocation` state variable and related control logic. The process begins when a user triggers the `setDeliveryAddress` function to set their address. In this reasonable situation, the invocation of `wx.getLocation` is rational. Upon success, its callback sets the `hasLocation` flag to `true` (line 10). However, users also have the right to choose to refuse manually setting the address in the *fail* function (line 12). Subsequently, in the logic layer (`.js`), an `if` statement (line 16) gates the `wx.navigateTo` invocation (line 17) on the `hasLocation` flag. Similarly, in the view layer (`.wxml`), a promotional banner’s visibility is made contingent on the same flag using the *wx:if* directive (line 4). In essence, this design abuses the permission obtained in a reasonable situation (setting an address) to gate an entirely unrelated functionality.

Case 2 — Functional Restriction tied to Sensitive User Input. As shown in Figure 2b, This case demonstrates a functional restriction triggered by sensitive user input. The MiniApp renders a menu list only after the user provides a valid phone number. Initially, the phone state variable is an empty string (line 3). A `<view>` component containing the menu is conditionally rendered via *wx:if*=`"phone"` (line 3), which initially evaluates to false, hiding the content. The `onPhoneInput` event handler validates the user’s input. Only upon successful validation does the MiniApp update the phone state via `setData()` (line 8), which satisfies the *wx:if* condition and immediately renders the previously hidden menu. The design thereby makes the disclosure of a phone number a prerequisite for accessing the menu content, despite this data being unnecessary for viewing a menu.

Challenges in Detecting Indirect Violations. Detecting Indirect Violations is challenging for existing analysis frameworks because they fail to address three fundamental issues:

1. Incomplete Modeling of Privacy Sources. Indirect Violations can be triggered by multiple sources: not only by privacy-related API invocations but also by sensitive data from user inputs (e.g., forms). Detectors that limit their scope to model only API-based sources will inevitably miss violations where functionality is gated on sensitive user input.

2. Overlooking Control-Flow Dependencies. The logic behind an instance of Indirect Violation relies on control-flow dependencies tied to a privacy information. For instance, after a user denies a permission request, a boolean flag is set. This flag then dictates the outcome of a conditional statement that blocks access to a feature. While a purely data-flow-oriented analysis might see the flag’s value change, it is typically blind to the consequence of this change on the program’s execution path. Thus, existing tools fail to connect the user’s privacy decision to the subsequent functional restriction.

3. Ignoring Functional Restriction. In cases of Indirect Violation, the harm is not data leakage but a denial of functionality. This adverse outcome occurs at specific program points that conditionally block user access. In some cases, the framework restricts navigation by gating invocations to APIs like *wx.navigateTo*. In other cases, it restricts visibility through a *setData* invocation that updates the state variable bound to a conditional rendering directive (e.g., *wx:if*)[11]. Because these invocations do not involve sending data over the network, tools designed exclusively to find information leakage vulnerabilities completely ignore them. They fail to recognize that restricting functionality is an effective, and often more subtle, method of coercing data disclosure.

3.2. Motivating Cases: Irrational Purposes

We present three real cases where the declared purpose of privacy information violated data protection principles.

Case A — Purpose Anomaly. Declaration: location is used for identity verification during login.

Analysis: this declared purpose is not consistent with common industry practices. Consequently, because location data is both non-unique and transient,

```

1: // code in page_shop.js
2: Page({
3:   data: {
4:     hasLocation: false
5:   },
6:
7:   setDeliveryAddress() {
8:     wx.getLocation({
9:       success: (res) => {
10:        this.setData({ hasLocation: true });
11:        setLocation(res) },
12:       fail: () => {inputAddress();}
13:     });
14:   },
15:   getProductDetail(){
16:     if(this.data.hasLocation){
17:       wx.navigateTo({ url: '/pages/product' });
18:       return;
19:     }
20:     wx.showModal({title: 'need location'});
21:   }
22: });

1: <!-- code in page_shop.wxml -->
2: <button bindtap="setDeliveryAddress">Set Address</button>
3: <button bindtap="getProductDetail">View Product</button>
4: <view wx:if="{{hasLocation}}">Free Shipping Over $29</view>

```

(a) Functional restriction contingent on a privacy-related API call.

```

1: // code in menuList.js
2: Page({
3:   data: { phone: "" },
4:
5:
6:
7:   onPhoneInput(e) {
8:     const value = e.detail.value.trim();
9:     if (/^[3-9]\d{9}$/.test(value)) {
10:      this.setData({ phone: value });
11:    }
12:  }
13: });

1: <!-- code in MenuList.wxml -->
2: <view class="menu-list">
3:   <text>Milk Tea</text>
4: </view>
5:
6:
7:
8: <view wx:if="{{!phone}}" class="mask">
9:   <input bindinput="onPhoneInput"/>
10: </view>

```

(b) Functional restriction contingent on sensitive user input.

Figure 2: Motivating examples of Indirect Violation.

it is considered fundamentally unsuitable for identity verification. If users are required to disclose their location to complete login, this practice is likely to result in unnecessary collection of sensitive information. This unnecessary collection constitutes a violation of the data minimization principle.

Case B — Purpose Ambiguity. Declaration: camera is used for taking photos.

Analysis: the MiniApp offers several distinct functionalities that use the camera (e.g., product review, AR try-on). However, the policy does not associate data collection with any concrete purpose, preventing users from knowing the specific purpose in advance. This declaration contravenes the principle of transparency.

Case C — Replaceable Redundancy. Declaration: ID number is used for account registration.

Analysis: the collection of an ID number is unnecessary. The application should instead use phone number registration, which achieves the same functional outcome with less privacy intrusion. This practice violates the necessity requirement of the data minimization principle.

4. Methodology

4.1. Framework Overview

Our methodology is operationalized through the MiniEval framework, whose high-level architecture is illustrated in Figure 3. The framework is organized into two primary components. The first, the Compliance Violation Detection Module operates on two distinct dimensions: Behavioral Violation Detection and Purpose Rationality Classification. The second component, Privacy Risk Assessment Module, then quantifies the identified risks into a single score and a risk level.

The Behavioral Violation Detection Module is responsible for identifying violations of principles based on the MiniApp’s code and behavior. This sub-module itself follows a three-stage process:

1. **MiniApp Exploration and Code Acquisition:** The framework first conducts dynamic analysis to systematically explore the MiniApp to obtain its complete source code.

2. **Constructing Violation Mappings via Static Analysis:** Next, Static Analysis processes the acquired code to identify potential violations. These results are then abstracted into two high-level mappings: the Privacy_Page Mapping (M_{pp}) and the Privacy_Constraint Mapping (M_{pc}), which are defined in Section 4.2.2.
3. **Violation Verification:** The framework employs a hybrid verification process. First, direct discrepancy matching between the code and the policy is performed to detect Omitted and Redundant Disclosure. For more nuanced cases requiring semantic understanding, a LLM is then used for deeper verification, which is twofold:
 - For potential direct violations (from M_{pp}), it performs Scenario Consistency Verification to confirm if the actual usage scenario aligns with the declared scenario.
 - For potential indirect violations (from M_{pc}), it performs Functional Necessity Assessment to determine if a functional restriction is justified.

The second module, the Purpose Rationality Classification Module, evaluates the declarative dimension of the data protection principles. It employs a LLM to determine whether the declared purposes in privacy policies are rational, classifying irrational purposes into three categories: Purpose Anomaly, Purpose Ambiguity, and Replaceable Redundancy.

The final module, the Privacy Risk Assessment Module derives an aggregated privacy risk score based on violation severity, purpose rationality, and data sensitivity. This score is then mapped to a corresponding risk level, providing a quantitative measure of the MiniApp’s privacy posture.

4.2. Behavioral Violation Detection

4.2.1. App Exploration and Code Acquisition

We use Dynamic Exerciser [12], a computer-vision-driven crawler, to autonomously traverse MiniApp’s UI and retrieve all sub-packages. Subsequently, we utilize Unveiler[13] to unpack these packages, yielding the complete and structured source code required for our static analysis engine. Concurrently, we employ Frida [14] to monitor the MiniApp’s behavior during

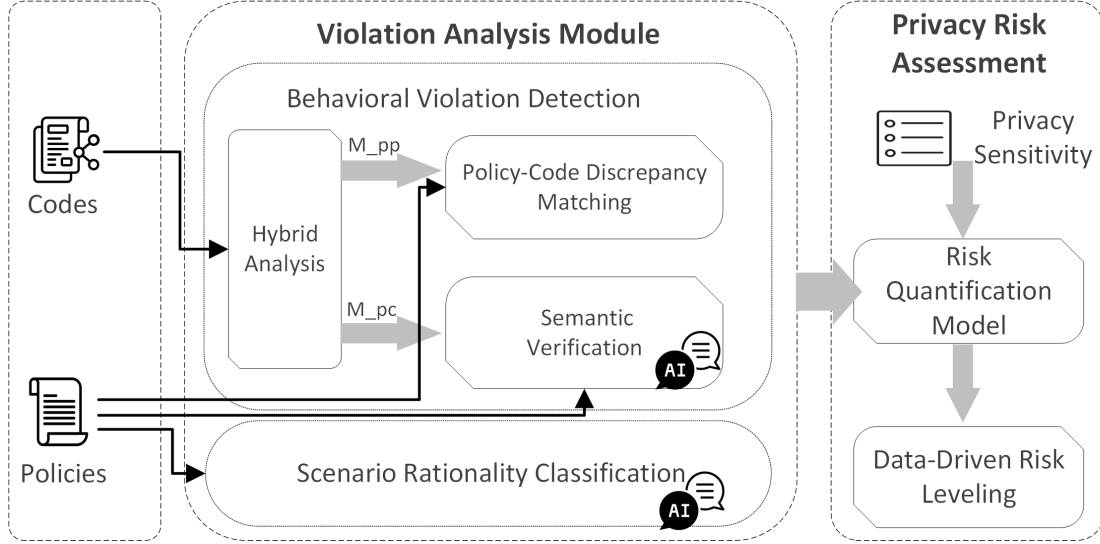


Figure 3: The Overall Workflow of MiniEval

the crawling process. By hooking into the runtime, Frida logs all sensitive API invocations. These logs provide an essential ground truth that is later used to validate and complement our static analysis results.

4.2.2. Constructing Violation Mappings via Static Analysis

Our static analysis constructs two structured artifacts that underpin violation detection: the Privacyx_Page Mapping (M_{pp}) and the Privacy_Constraint Mapping (M_{pc}). Formally,

$$M_{pp} : PrivacyData \rightarrow Page \quad (1)$$

$$M_{pc} : PrivacyData \rightarrow RestrictedComponent \text{ or } Page \quad (2)$$

For M_{pp} , each privacy data item is associated with the MiniApp page where it is used. For M_{pc} , each privacy data item is associated with the the UI component or page whose accessibility is constrained by that data.

Constructing Privacy_Page Mapping. For M_{pp} , we directly adopt the approach of Wang et al. [15], who proposed the Do-as-You-Say framework for consistency detection between MiniApp code and privacy policies. Their method treats both privacy APIs and user inputs as data sources, and builds a Data Dependency Graph to trace how these data propagate within each page. By associating each data practice with the corresponding `page.js`

and `page.wxml` files, they construct a mapping from privacy data items to the MiniApp pages where they are used. We use their results to construct M_{pp} .

Constructing Privacy_Constraint Mapping. To systematically build M_{pc} , our approach (i) identifies the universe of privacy data and the potential functional restrictions, (ii) models control dependencies and data dependencies, and (iii) applies Control-Aware Privacy Flow Analysis (CAPFA) to extract the dependency paths and use them to construct M_{pc} .

identify privacy data and potential functional restrictions. Before running CAPFA, we identify the analysis endpoints for our backward traversal. Functional restrictions serve as the entry points for the traversal, while privacy data serve as its termination points.

- **Identify privacy data.** Our privacy data model is comprehensive, encompassing data from two primary sources: privacy-related APIs and sensitive user inputs. We identify each source through a specific approach. API invocations are identified directly by traversing the Abstract Syntax Tree (AST). For user inputs, we identify sensitive data by matching privacy-related keywords with regular expressions.
- **Identify potential functional restrictions.** For restrictions of navigation APIs, such as `wx.navigateTo`, we identify their invocations during AST traversal and mark them as potential functional restrictions. For restrictions enforced by `setData` invocations, we adopt a two-phase approach. First, we scan the `.wxml` files using regular expressions. This step extracts state variables related to conditional UI rendering (e.g., those governed by `wx:if`). Next, we analyze the corresponding `.js` files by traversing their ASTs. In this phase, we identify `setData` invocations that modify the extracted variables. As a result, by combining these two steps, we can accurately identify potential functional restrictions in conditionally rendered UI components

Modeling Control Dependencies. To resolve this, we extend MiniTracker’s Assignment Flow Graph (AFG) into a Control-Enhanced Assignment Flow Graph (CEAFG). The AFG [16] model represents a program as a graph, where nodes represent program states and edges represent data flows

based on assignment. It has established the data dependencies required for analysis. However, an inspection of the implementation of MiniTracker reveals that its handling of control dependencies is only partial.

For the code in Listing 1, MiniTracker identifies the dependency from *condition* to *foo*, but missing the *condition* to *bar*, because it only handles control dependencies between adjacent program states.

```

1 if (condition == true) {
2     foo = 1; // Statement 1
3     bar = 2; // Statement 2
4 }
```

Listing 1: Example for control-dependency coverage

To address this, we traverse the AST. During this process, conditional predicates (e.g., if conditions) connect to all program states within their body scopes. Likewise, privacy API invocations connect to all program states within the scopes of their success and fail callback functions. In both cases, we construct the dependency edges at the scope level. This approach ensures that every program state is directly linked to the predicate or API invocation that governs it. By incorporating these comprehensive control dependencies into the AFG, we obtain the CEAFG, formally defined as $G = (V, E_d, E_c)$.

Implementation of the Control-Aware Privacy Flow Analysis Algorithm. CAPFA is designed to identify indirect violation paths by traversing the CEAFG, a graph structure that unifies both data and control dependencies. As shown in Algorithm 1, our analysis defines two key endpoints. The functional restrictions serve as the traversal sinks, while the privacy data act as the sources. The algorithm then performs a backward traversal from these sinks to the sources. During traversal, the algorithm maintains a worklist of nodes to visit, a done set of visited nodes, and a dependency set. The traversal operates as follows. The worklist is initialized with all sink nodes (line 3). The algorithm then iteratively processes nodes from the worklist until it is empty (line 4). For each node to be processed, the algorithm retrieves all its control or data dependencies; It records each dependency edge and adds the corresponding predecessor node to the worklist for processing in a subsequent iteration. (lines 7-18). Once traversal terminates (i.e., the worklist is empty), a DFS-based `generate_paths` function

Algorithm 1 Control-Aware Privacy Flow Analysis

```
1: Input: ceafg, sources, sinks
2: Output: iv_paths
3: worklist  $\leftarrow$  sinks, done  $\leftarrow \emptyset$ , dep  $\leftarrow \emptyset$ 
4: while worklist  $\neq \emptyset$  do
5:   current_node  $\leftarrow$  worklist.pop()
6:   done  $\leftarrow$  done  $\cup$  {current_node}
7:   for each pred  $\in$  get_predecessors(ceafg, current_node) do
8:     if pred  $\notin$  done then
9:       if is_control_dep(pred, current_node) then
10:        worklist  $\leftarrow$  worklist  $\cup$  {pred}
11:        dep  $\leftarrow$  dep  $\cup$  {(current_node, pred)}
12:      end if
13:      if is_assignment(pred, current_node) then
14:        worklist  $\leftarrow$  worklist  $\cup$  {pred}
15:        dep  $\leftarrow$  dep  $\cup$  {(current_node, pred)}
16:      end if
17:    end if
18:  end for
19: end while
20: iv_paths  $\leftarrow$  generate_paths(dep, sinks, sources)
```

generates all paths from sinks back to sources from the collected dependency edges (line 20). We then construct the M_{pc} by mapping each dependency path back to the concrete functional restriction that served as its sink. The specific information recorded in M_{pc} depends on the type of restriction:

- For navigation APIs, we first apply alias analysis to resolve its url attribute. We then retrieve the *.xml* source code of the resolved page URL and record this code as the representation of the restriction.
- For UI components, we record the full implementation context of the restricted component. This includes its *.xml* snippet, its associated JavaScript event handlers, and the *.xml* code of the page that contains the component.

4.2.3. Detection of Direct Violations

Our framework systematically identifies three types of direct violations: Redundant Disclosure, Omitted Disclosure and Scenario Inconsistency. The detection process combines straightforward discrepancy with advanced LLM-based analysis. This entire stage is guided by the `M_pp`, which serves as a manifest to extract the necessary code and policy statements for comparison. Specifically, the discrepancy check is sufficient for identifying the presence or absence of data practices (Omitted and Redundant Disclosure), while the more nuanced task of detecting Scenario Inconsistency necessitates the contextual understanding provided by the LLM.

Policy-Code Discrepancy Matching. The detection of Omitted and Redundant Disclosure is performed via a direct comparison between the declared data items and the collected items cataloged in our `M_pp`. This comparison identifies two types of violations. Redundant Disclosure occurs when a declared item is absent from `M_pp`, while Omitted Disclosure occurs when an item present in `M_pp` is not found in the policy. To ensure the precision of our `M_pp`, especially for user input information, we employ a LLM-based verification step. While our static analysis uses regular expressions to identify all potential input-related privacy data, this can introduce semantic false positives (e.g., a match occurring within a static text label rather than an input field). Therefore, before conducting the comparison, we use a LLM to analysis the code context of each regex-identified candidate. The LLM verifies whether the element represents an interactive component designed for data collection. This approach effectively filtering out false positives and ensuring the high accuracy of detection.

Verifying Scenario Consistency via Semantic Analysis. The detection of Scenario Inconsistency aims to determine whether a MiniApp’s actual usage scenario aligns with its declared scenario. Achieving this requires a deep semantic understanding of the page’s code, for which we leverage an LLM. Guided by the `M_pp` mapping, the LLM performs a two-step assessment for each page: (i) Usage Scenario Identification. In this step, the LLM analyzes the `.wxml` source code of the page to identify the actual usage scenario. (ii) Consistency Verification. In this step, the LLM compares this identified scenario against the declared one to determine alignment.

To guide the LLM in this detection effectively, the design of our prompt incorporates several techniques:

- **Role-Playing and Clear Task Definition:** For Step 1, we tasked the LLM with inferring the usage scenario of the page from the code. For Step 2, we instruct the LLM to act as a professional privacy auditor and clearly define its task: to determine whether the page’s usage scenario aligns with the declaration.
- **Contextual Grounding:** The prompt provides the LLM with all necessary context in a structured format: the corresponding declaration and the `.wxml` code representing the page’s actual usage scenario.
- **Structured Output:** The LLM is required to output its conclusion in a strict JSON format (`{"is_consistent": boolean, "reason": "..."}`), which allows for easy programmatic parsing.

Specifically, our method for detecting SI offers notable advantages in scalability and completeness over prior works. While prior approaches like Shenao Wang et al. [8] also leverage a LLM, their reliance on dynamic exploration and OCR of screenshots can be slow and may fail to achieve full page coverage. In contrast, our method directly retrieves the complete source code via static analysis. This provides the LLM with a structured representation of the entire UI, ensuring a more efficient and robust consistency check.

4.2.4. Detection of Indirect Violations

The detection of Indirect Violations aims to determine whether a functionality restriction tied to a privacy item is functionally reasonable. This judgment requires a deep semantic understanding of the restricted functionality, a task for which we employ an LLM. For each potential violation, the LLM performs a two-step assessment based on the related source code recorded in `M_pc`: (i) Functionality Summarization. In this step, the LLM analyzes the related source code of the restricted element to generate a summary of its actual functionality. (ii) Necessity Analysis. In this step, the LLM determines if the privacy item is necessary for the summarized functionality.

To guide the LLM in this detection effectively, the design of our prompt incorporates several techniques:

- **Role-Playing and Clear Task Definition:** For Step 1, we instruct the LLM to act as a code summarizer. For Step 2, it is instructed to act as a privacy auditor to assess the necessity.

- **In-Context Learning with Examples:** Crucially, for the necessity assessment, it provides the LLM with a formal definition of Indirect Violation and a series of justified and unjustified few-shot examples (from Section 3.1).
- **Structured Output:** The LLM must return its verdict in a strict JSON format (`{"is_reasonable": boolean, "reason": "..."}).`

This LLM-driven analysis of functional necessity allows our framework to move beyond simple path detection and systematically identify cases where users are subtly coerced into providing unnecessary data.

4.3. Purpose Rationality Classification

The Purpose Rationality Classification module performs a semantic audit of privacy policies. Its objective is to evaluate whether a declared purpose for data collection is rational and adheres to data protection principles. Specifically, it identifies three types of irrationality previously detailed in Section 3.2: *Purpose Anomaly*, *Purpose Ambiguity*, and *Replaceable Redundancy*. This classification is performed by an LLM guided by a carefully engineered prompt. For each declared purpose, the prompt provides the LLM with a clear task definition, the formal definitions of the three irrationality types, and few-shot examples of each category (see Section 3.2 for examples). The LLM is then required to return its output in a structured JSON format, which is shown below:

```
{
  "is_reasonable": true | false,
  "violation_type": "Purpose Anomaly" | "Purpose Ambiguity" |
  "Replaceable Redundancy" | "None",
  "reason": "..."
```

Through this LLM-driven approach, our framework can perform a semantic audit of privacy policies, scrutinizing the validity of declared purpose.

4.4. Privacy Risk Assessment

The final module of our framework, the Privacy Risk Assessment Module, synthesizes the outputs from the preceding analysis stages to compute a Privacy Risk Score and a risk level.

4.4.1. A Multi-Level Sensitivity Taxonomy for Privacy Information

A cornerstone of our risk assessment is a multi-level sensitivity classification standard. To ensure objectivity and rigor, our classification is guided by established laws and regulations, including China’s Personal Information Protection Law (PIPL)[3] and national standards such as GBT45574-2025[17]. Building upon legal foundations and empirical insights, we propose a practical four-level hierarchical scheme to assess the sensitivity of privacy information. Our classification and their corresponding sensitivity weights, $W_{sens}(p)$, are defined as follows:

- **Level 4: Critical Sensitivity:** Data leakage at this level could endanger personal and property safety. Examples: *Precise Geolocation, Contact Lists, National ID Numbers, Biometric Data, Financial and Health Records.*
- **Level 3: High Sensitivity :** This level includes data that, while highly sensitive, may require more context for severe exploitation. This level typically includes data that allows for the creation of detailed user profiles or reveals sensitive personal behaviors and social connections. Examples: *User-Generated Content (e.g., posts, media files) and Transaction History.*
- **Level 2: Medium Sensitivity:** This level includes data that contributes to a user’s online identity or social profile but does not directly expose their real-world identity. Examples: *User Nickname, Avatar and Gender.*
- **Level 1: Low Sensitivity:** This level consists of technical or device-related data that poses minimal direct privacy risk. Examples: *Device Model, Operating System Version, Network Status, and specific Device Identifiers (e.g., IMEI, MAC Address).*

This standard provides the formal foundation for our weighted risk scoring model. A complete mapping of all analyzed privacy items to these levels is detailed in our online documentation[10].

4.4.2. Risk Score Formulation and Parameter Definition

Building upon this sensitivity classification, we formulate the Privacy Risk Score (PRS), a quantitative metric designed to capture the absolute magnitude of a MiniApp’s privacy risk. We intentionally designed the PRS

as a cumulative score rather than a percentage-based rate. Percentage metrics, even when weighted, can be fundamentally misleading as they obscure the underlying risk scale. For instance, consider two MiniApps, each with one violation out of two data practices. If MiniApp A’s practices involve two low-sensitivity items while MiniApp B’s involve two critically sensitive ones, both would yield an identical 50% non-compliance rate. This misleading equivalence fails to represent higher risk in MiniApp B. Our cumulative approach avoids this flaw. The PRS is formally calculated using the following weighted linear model:

$$PRS = \sum_{p \in P_{app}} W_{sens}(p) \times (\gamma + \alpha \cdot V(p) + \beta \cdot R(p)) \quad (3)$$

Where:

- P_{app} is the set of all unique privacy items associated with the MiniApp.
- $W_{sens}(p)$ is the sensitivity weight assigned to a specific privacy item p .
- $V(p)$ is a binary flag indicating a detected behavioral violation associated with p .
- $R(p)$ is a binary flag indicating an irrational declared purpose for item p .
- γ, α, β are hyperparameters representing the weights for inherent risk, behavioral violation risk, and rationality risk, respectively.

This formulation operationalizes our risk assessment philosophy. The hyperparameter γ establishes a non-zero baseline risk for any data collection, acknowledging that even compliant data handling expands a user’s attack surface. The distinct weights for α and β allow the model to differentiate between the severity of concrete behavioral violations and declarative flaws in the privacy policy, providing a nuanced and multi-faceted privacy risk score.

4.4.3. Data-Driven Risk Leveling

While the PRS provides a granular, continuous score, practical applications in policy-making and user communication require a mapping to discrete, interpretable risk tiers (e.g., Low, High). The central challenge, therefore, is to define the boundaries for these tiers objectively. A manual approach is untenable, as it would be an inherently arbitrary and subjective process, lacking statistical validity and reproducibility.

To define these boundaries objectively, we employ clustering analysis, a data-driven methodology. The underlying rationale is that the statistical distribution of PRS scores is not uniform; instead, it contains an intrinsic structure. Clustering algorithms are designed precisely to uncover such structures. They operate by partitioning data to maximize intra-cluster similarity (cohesion) while minimizing inter-cluster similarity (separation). The resulting boundaries can be interpreted as natural "break points" in the distribution, signaling significant shifts in the underlying risk profile. This approach ensures our classification is objective, reflecting the data's inherent structure rather than subjective judgment. The empirical validation of this methodology and the selection of the optimal clustering algorithm are detailed in our evaluation (Section 5.3).

5. Evaluation

In this section, we present a series of experiments designed to comprehensively evaluate our framework, MiniEval. Our evaluation is structured to answer three central research questions:

- **RQ1: Effectiveness of Compliance Violation Detection.** How effectively does MiniEval detect violations of data protection principles in the behavioral and declarative dimensions?
- **RQ2: Soundness of Quantitative Risk Assessment.** How sound is the methodology behind MiniEval's Privacy Risk Assessment model for quantitatively classifying privacy risks?
- **RQ3: Real-World Privacy Landscape.** What is the landscape of privacy risks in the MiniApp ecosystem and what does our risk assessment reveal about the severity and distribution of violations in different application categories?

5.1. Experimental Setup

Dataset. Our study leverages a large-scale dataset of 15,917 WeChat MiniApps, collected by Yang et al.[18]. This dataset provides the foundation for our large-scale study on privacy risks in the MiniApp ecosystem (RQ3). We found 12,365 MiniApps (77.7%) either lacked a privacy policy or declared that they had not collected private information. This finding points to a systemic disregard for transparency and prompts our comprehensive audit of the full spectrum of privacy violations.

Ground Truth. To establish a reliable ground truth for evaluating MiniEval’s effectiveness (RQ1), two researchers with expertise in MiniApp privacy analysis independently inspected a randomly selected subset of 350 MiniApps. They analyzed the source code, UI, and privacy policy of each app to identify all instances of direct violations, indirect violations, and purpose irrationality.

Implementation and Environment. MiniEval’s static analysis engine is implemented in JavaScript. Its LLM-based verification module utilizes the `qwen-plus-2025-07-14` model, invoked with a `temperature` of 0.2 to ensure output stability. All experiments were conducted on a machine running Ubuntu 22.04, equipped with an Intel Core i5-8300H CPU @ 2.30GHz and 16GB of RAM.

5.2. RQ1: Effectiveness of Compliance Violation Detection

To answer RQ1, we evaluate MiniEval’s effectiveness on our manually labeled ground truth. The evaluation is structured to highlight the novel analytical dimensions that MiniEval introduces beyond prior work like MiniScope [7], proceeding in three stages: First, we assess its performance in detecting Direct Violations, particularly its ability to analyze user input sources and identify Scenario Inconsistency. Second, we quantify its effectiveness in detecting Indirect Violations. Finally, we evaluate its unique capability to identify unreasonable purposes.

5.2.1. Effectiveness in Detecting Omitted and Redundant Disclosure

Our effectiveness evaluation for direct violations begins with the results presented in Table 1.

Baseline Comparison for Omitted and Redundant Disclosure. We first establish a baseline on API-based privacy data by comparing MiniEval to MiniScope. As shown in Table 1, MiniScope demonstrates highly optimized performance on this specific task, achieving superior precision for both Omitted Disclosure and Redundant Disclosure.

MiniEval’s lower precision on API sources is an expected consequence of a design trade-off: our analysis forgoes full path reachability analysis in favor of a broader scope that includes user inputs and control-flow dependencies. While this leads to a higher number of false positives from unreachable code, MiniEval still achieves a reasonable recall, confirming that its foundational analysis capabilities are sound.

Uncovering the Blind Spot of User-Input Issues. The core advancement of MiniEval is its ability to analyze issues originating from user input, a critical blind spot for existing frameworks. As shown in Table 1, MiniScope fails to detect any such issues, resulting in a 0% recall. Specifically, it missed all 394 instances of OD and 34 instances of RD.

MiniEval’s two-stage (Regex+LLM) pipeline is designed to address this challenge. The first stage, our MiniEval-Regex configuration, acts as a high-recall candidate generator. It successfully achieves a recall of 86.8% for user-input OD, but its inherent limitations in pattern matching result in 52 false negatives. These primarily stem from its inability to process text embedded in image assets or to identify keywords in obfuscated variables.

Simultaneously, the regex-based stage suffers from semantic errors due to its inability to distinguish context. It frequently misidentifies static descriptive text as interactive input fields and incorrectly flags non-private identifiers due to partial matching. These errors directly impact performance. For OD, they produced 98 false positives, lowering the precision to 77.7%. More subtly, for Redundant RD, the same misinterpretation of static text as a valid data use caused two false negatives.

The final MiniEval-LLM stage acts as a semantic filter designed precisely to address these errors. For user-input OD, it boosts precision from 77.7% to 98.0% by eliminating 91 false positives. For user-input RD, it corrects the misinterpretations, achieving an F1-score of 100% by resolving the two false negatives.

5.2.2. Effectiveness in Detecting Scenario Inconsistency

MiniEval detects Scenario Inconsistency, a nuanced type of violation where the context of data collection deviates from the declared purpose. Automating the detection of these semantic mismatches remains a significant challenge for prior tools. MiniEval addresses this with a dedicated, LLM-based verification module.

To quantify its performance, we evaluated the SI detection module on all relevant instances in our ground truth set. The results are detailed in Table 2. The module achieves an F1-score of 88.9%, supported by a precision of 92.3% and a recall of 85.7%. This validates our LLM-driven approach for reliably automating the detection of these complex, context-dependent issues.

Analysis of Misclassifications. The root cause of these misclassifications is the build process employed by third-party frameworks like Uni-app, which

Table 1: Effectiveness Comparison for Omitted Disclosure (OD) and Redundant Disclosure (RD) Detection on our Ground Truth of 350 MiniApps. Pre% represents Precision%; Rec% represents Recall%; F1% represents F1-score%.

Category	Tool	GT	TP	FP	FN	Pre%	Rec%	F1%
OD (API)	MiniScope [7]	579	571	21	8	96.5%	98.6%	97.5%
	MiniEval-LLM	579	572	52	7	91.7%	98.8%	95.1%
OD (Input)	<i>MiniEval-Regex</i>	394	342	98	52	77.7%	86.8%	82.0%
	MiniEval-LLM	394	342	7	52	98.0%	86.8%	92.1%
OD (Total)	MiniEval-LLM	973	914	59	59	93.9%	93.9%	93.9%
RD (API)	MiniScope [7]	202	199	0	3	100%	98.5%	99.2%
	MiniEval-LLM	202	181	0	21	100%	89.6%	94.5%
RD (Input)	<i>MiniEval-Regex</i>	34	30	4	4	100%	94.1%	97.0%
	MiniEval-LLM	34	33	4	1	100%	100%	100%
RD (Total)	MiniEval-LLM	236	215	0	21	100%	91.1%	95.3%

reorganizes and bundles the source code into modularized utility components.

This transformation systematically obfuscates static call chains. Specifically, when a privacy api invocation is encapsulated within a bundled module, our static analyzer may fail to resolve the page that uses the api. This initial detection failure prevents the analyzer from retrieving the complete pages, thereby providing the LLM with insufficient information for the SI verification stage and causing misclassifications.

Table 2: Effectiveness of MiniEval’s Module for Scenario Inconsistency (SI) Detection. The 14 ground truth (GT) instances were manually identified from the 110 declarations in our 350-app set that passed the initial OD/RD screening. Pre% represents Precision%; Rec% represents Recall%; F1% represents F1-score%.

Violation Type	GT	TP	FP	FN	Pre%	Rec%	F1%
Scenario Inconsistency	14	12	1	2	92.3%	85.7%	88.9%

5.2.3. Effectiveness in Detecting Indirect Violations

Indirect Violations—where access to functionality is coercively conditioned on non-essential data disclosure—represent a subtle privacy issue not addressed by prior automated tools. To validate MiniEval’s effectiveness

in detecting this novel issue, we performed a two-part evaluation using our ground truth set and a larger set of real-world candidates.

Quantitative Evaluation on Ground Truth. First, we conducted a rigorous evaluation on all 15 instances of Indirect Violations within our manually labeled ground truth set. As detailed in Table 3, MiniEval identified 11 of these cases, achieving an F1-score of 81.5%. This result is supported by a precision of 91.7% (11 of 12 positive findings were correct) and a recall of 73.3% (11 of 15 true instances were found).

Precision Validation on Real-World Candidates. To further validate the reliability of our detection pipeline, we performed a precision-focused analysis on a diverse set of real-world examples. We first applied our two-stage detection strategy to the full 15,917-app dataset.

The initial static analysis engine efficiently generated **2,291** potential **Privacy-Constraint Mappings** (as defined in Section 4.2.2), linking privacy data denials to potential functional restrictions. These mappings were then passed to our LLM module for a deeper, semantic assessment of functional necessity.

The LLM filtered out a significant number of these initial mappings, ultimately flagging **689** as high-confidence candidates of Indirect Violations. A manual review of the discarded cases revealed that the majority were indeed functionally justified restrictions. For example, a common discarded case involved an image-editing feature that required access to the photo library; if the user denied permission, the app would correctly disable the image display component. By filtering out such legitimate dependencies, the LLM demonstrated its ability to distinguish between coercive practices and necessary functional constraints.

From the 689 high-confidence candidates, we drew a random sample of 80 for manual expert review. Our analysis confirmed that 73 were true instances of Indirect Violation, demonstrating a high precision of 91.2%. This result confirms that MiniEval’s positive identifications are highly reliable when applied at scale. To complement this precision analysis, we next investigated the sources of recall limitations by examining the false negatives.

Error Analysis of Recall Limitations. A qualitative analysis of the false negatives reveals that our recall limitations primarily stem from a semantic gap between our static analyzer and the architectural abstractions introduced by third-party development frameworks (e.g., `uni-app`). Our

analyzer is designed to parse native MiniApp syntax, whereas these frameworks introduce proprietary patterns that obscure privacy-sensitive operations. This challenge manifests in two principal ways:

First, these frameworks employ unique abstractions for event handling and data binding. Our analyzer, modeled on native APIs, fails to recognize these non-native patterns as sources of user-provided data. Consequently, it under-reports violations that originate from these abstracted input channels.

Second, these frameworks are often coupled with centralized state management libraries, such as Vuex. We observed numerous cases where the url parameter for navigation APIs (e.g., `wx.navigateTo`) was sourced from a global state store. The retrieval occurs via framework-specific APIs (e.g., `mapState` or `this.$store.state`), for which our analysis model currently lacks the necessary semantics. This semantic gap prevents our analyzer from performing the required alias analysis to resolve the concrete URL path. As a result, it cannot identify violations contingent on navigation to a specific page, thus contributing to false negatives.

Table 3: Effectiveness of MiniEval’s Module for Indirect Violations Detection. The [GT] ground truth (GT) instances were manually identified from our ground truth set of 350 MiniApps. Pre% represents Precision%; Rec% represents Recall%; F1% represents F1-score%.

Violation Type	GT	TP	FP	FN	Pre%	Rec%	F1%
Indirect Violation	[15]	[11]	[1]	[4]	[91.7%]	[73.3%]	[81.5%]

5.2.4. Effectiveness in Detecting Irrational Declared Purposes

We evaluate the effectiveness of our Purpose Rationality Classification Module. This module scrutinizes privacy policies to identify three categories of irrational purpose declarations: Purpose Ambiguity, Purpose Anomaly, and Replaceable Redundancy. To enhance classification stability, we query the LLM three times for each case and use a majority vote for the final prediction.

Table 4 details the performance of the Purpose Rationality Classification module on our manually labelled dataset. The results demonstrate a high level of effectiveness, with a macro-averaged F1-score of 90.5% achieved across all irrationality types.

The model performed particularly well at identifying Purpose Ambiguity, the most prevalent issue type, achieving an F1-score of 92.2%. It also per-

formed robustly for Purpose Anomaly and Replaceable Redundancy, achieving F1-scores of 87.0% and 92.3%, respectively. The model’s positive identifications are highly reliable. Across all categories, the precision ranged from 87.0% to 100%.

Analysis of Misclassifications. Our error analysis of the few misclassified cases indicates two primary sources of error.

The false negatives are primarily due to the subtle semantic overlap between the categories. For instance, the three false negatives for Purpose Anomaly (PA) were misclassified as Purpose Ambiguity (PB). This suggests that while the model correctly identified an issue with the declaration, it struggled with the fine-grained distinction between a "vague" purpose and an "anomalous" one in borderline cases.

Conversely, a portion of the false positives stemmed from the LLM’s tendency to be overly-strict in its interpretation. We observed cases where the model flagged a purpose declaration as ambiguous (PB) because it lacked absolute, fine-grained specificity, even though a human reviewer might consider it acceptable within the broader context. This highlights an inherent challenge in automated policy scrutiny: balancing the need for precise language with the practical realities of policy writing.

Despite these borderline cases, the strong overall results validate that our LLM-based approach, guided by a well-structured prompt, can reliably and accurately scrutinize the fundamental rationality of privacy declarations.

Table 4: Effectiveness of Purpose Rationality Classification. Pre% represents Precision%; Rec% represents Recall%; F1% represents F1-score%.

Irrationality Type	GT	TP	FP	FN	Pre%	Rec%	F1%
Purpose Ambiguity	74	71	9	3	88.8%	95.9%	92.2%
Purpose Anomaly	23	20	3	3	87.0%	87.0%	87.0%
Replaceable Redundancy	7	6	0	1	100%	85.7%	92.3%

5.3. RQ2: Soundness of Quantitative Risk Assessment

This section validates the rationale and statistical soundness of our Privacy Risk Assessment model, which translates complex privacy issues into a quantitative, interpretable score.

Principled Model Design and Configuration. The hyperparameters in our PRS formula are configured based on established data protection principles. We assign a higher weight to a detected behavioral violation ($\alpha = 1.0$) than to an irrational declared purpose ($\beta = 0.5$), reflecting the principle that concrete program behaviors carry greater intrinsic risk than declarative statements. We also apply a non-zero baseline risk ($\gamma = 0.1$) to every data collection act. This acknowledges that any collection, even when compliant, carries an inherent privacy cost by expanding the user’s attack surface. Furthermore, the sensitivity weights (W_{sens}) follow a geometric progression (1, 2, 4, 8). This design models the non-linear increase in potential harm associated with higher data sensitivity, ensuring that violations involving more critical data incur a disproportionately larger penalty.

Empirical Score Distribution Analysis. Applying the Privacy Risk Assessment model to our dataset yielded the continuous score distribution shown in Figure 4. The Kernel Density Estimate (KDE) plot reveals a highly right-skewed distribution with a long tail. This indicates that while most MiniApps pose a low-to-moderate risk, a significant minority exhibit extremely high-risk profiles. This empirical distribution, which confirms the model’s ability to provide nuanced risk stratification, forms the basis for our subsequent tiering methodology.

Validation of the Data-Driven Risk Levels. Our levels process began with a crucial pre-processing step: we identified and temporarily set aside the top 5% of scores (683 apps) to prevent these extreme outliers from distorting the main clustering analysis. We then performed a comparative evaluation of three distinct methods to partition the remaining 95% of the data: the baseline Quantile Method, K-Means clustering, and Jenks Natural Breaks clustering.

We first evaluated the baseline Quantile Method, which divides the data into equally sized groups. This method was deemed unsuitable for our skewed distribution, as it erroneously assigned identical score values into two different tiers, violating classification consistency. We then evaluated two model-based clustering algorithms, K-Means and Jenks Natural Breaks, assessing a range of tier counts (from 2 to 7) to find an optimal balance between statistical fit and interpretability.

As shown in Table 5, the evidence from both metrics strongly supports a four-tier classification as the most robust and interpretable choice. For the

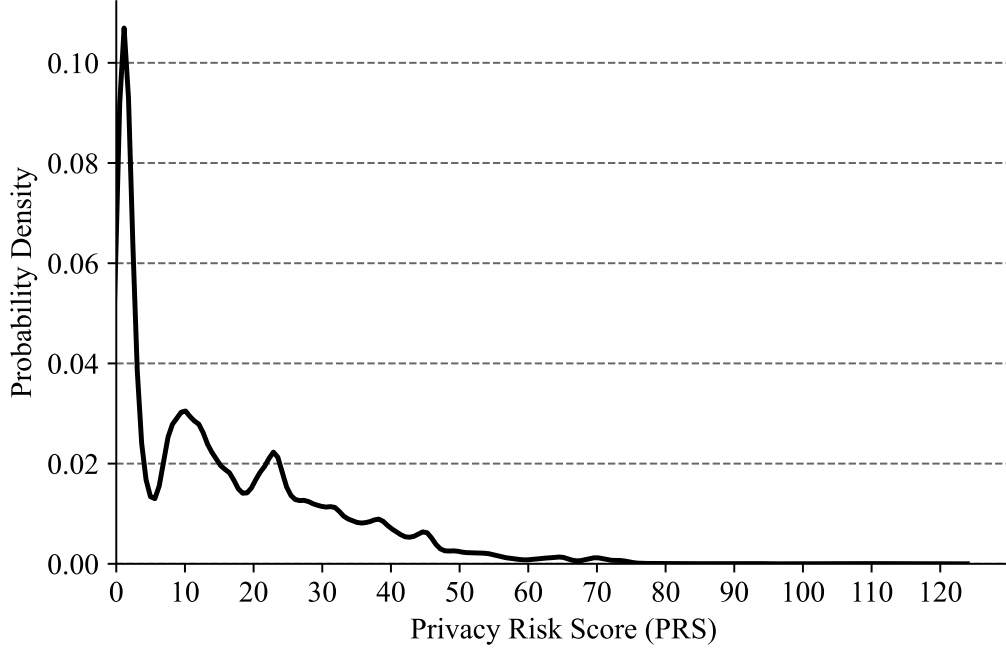


Figure 4: The distribution of the Privacy Risk Score is highly right-skewed. The plot shows the Kernel Density Estimate (KDE) of the scores, illustrating the continuous probability distribution

Jenks Natural Breaks analysis, the GVF metric exhibits a distinct "elbow" at four tiers (0.9484). Beyond this point, increasing the number of tiers yields only marginal improvements in statistical fit at the cost of increasing model complexity. Concurrently, while the Silhouette Score for K-Means technically peaks at $k = 6$ (0.7001), the score at $k = 4$ (0.6913) is comparably high. Critically, selecting the simpler $k = 4$ model avoids the risk of overfitting for a negligible performance gain, adhering to the principle of parsimony. Therefore, the combined evidence overwhelmingly supports $k = 4$ as the optimal trade-off between statistical fit and model simplicity.

Remarkably, when configured for four tiers, both algorithms produced an identical partitioning of the data, yielding the exact same score boundaries (Table 6). This perfect convergence provides powerful evidence that the identified four-tier structure is not an artifact of our chosen method, but rather reflects an intrinsic, objective grouping within the risk score data itself.

This rigorous methodology ensures our final risk tiers are both statistically sound and empirically grounded.

Based on this robust, validated partition, we establish our final four-tier risk classification. The first three tiers—**Low**, **Moderate**, and **High Risk**—are directly defined by the boundaries of the three lowest-risk clusters. The fourth and final tier, **Severe Risk**, is a composite category that consolidates the highest-risk cluster from our analysis with the top 5% of scores that were initially set aside. This methodology ensures that all extreme-risk profiles are definitively classified into our highest threat level, resulting in a comprehensive and empirically grounded four-tier framework.

Table 5: Model Selection Metrics for K-Means and Jenks Natural Breaks. Both metrics suggest that k=4 offers an optimal trade-off between model fit and simplicity.

Tiers (k)	Silhouette Score	GVF
2	0.6680	0.7504
3	0.6293	0.8858
4	0.6913	0.9484
5	0.6870	0.9678
6	0.7001	0.9792
7	0.6959	0.9845

Table 6: Convergence of Tiering Results between K-Means and Jenks. Both algorithms, at their optimal k=4 configuration, produced the exact same partitioning.

Tier Level	Score Range	App Count
Level 1 (Low)	0.00 – 6.60	5,749
Level 2 (Moderate)	6.60 – 17.60	4,395
Level 3 (High)	17.60 – 30.50	3,139
Level 4 (Severe)	30.50 – 45.10	1,951

5.4. RQ3: Real-World Privacy Landscape

Having validated the effectiveness of **MiniEval**’s detection modules (RQ1) and established its risk assessment framework (RQ2), we deployed it at scale to answer our final research question: *What is the landscape of data minimization issues in the wild, and what are the associated privacy risks?* To this end, we analyzed our entire dataset of 15,917 real-world WeChat MiniApps. Our findings paint a concerning picture of systemic non-compliance and reveal a risk landscape strongly correlated with application business models.

Prevalence of Privacy Issues. Our large-scale analysis reveals that apps declaring any data collection are a small minority; a mere 3,552 apps (22.3%) provide a privacy policy that explicitly states they collect user data. This immediately suggests a systemic failure in transparency, as many app categories (e.g., e-commerce) inherently require user data to function. This discrepancy points to a high prevalence of Omitted Disclosures and prompted a deeper investigation into the full spectrum of privacy issues:

- **Omitted Disclosure:** As anticipated, this was the most dominant violation, with a staggering 48,420 instances identified. The vast majority stemmed from MiniApps that provided no privacy policy whatsoever.
- **Other Direct Violations:** We identified 10,937 instances of Redundant Disclosure, indicating a widespread practice of over-declaring data collection. Additionally, 807 instances of Scenario Inconsistency were detected, where the actual data usage deviated from its declared purpose.
- **High-Risk Declarations:** Scrutinizing policy texts, we identified 4,929 instances of high-risk or irrational purpose declarations. The most frequent type was Purpose Ambiguity (PB) with 3,351 instances. While not direct violations, these declarations undermine the principle of purpose limitation.
- **Coercive Practices:** Finally, our analysis identified 689 high-confidence instances of Indirect Violations. This represents a particularly insidious form of non-compliance, often invisible to traditional static analyzers. In these cases, the MiniApp programmatically gates access to core functionality, making it conditional upon the user’s disclosure of non-essential data. This practice creates a coercive dynamic, effectively forcing users to trade their privacy for functionality and fundamentally undermining the principle of free and informed consent.

Quantitative Analysis of the Risk Landscape. Leveraging our established s Privacy Risk Assessment model and the four-tier classification scheme, we analyzed the quantitative risks across the ecosystem. We found that a combined 36.2% of MiniApps fall into the **High** or **Severe Risk** tiers, indicating a widespread problem.

Further analysis revealed a strong correlation between a MiniApp’s functional category and its privacy risk. Table 7 presents key metrics for each

category, while Figure 5 visualizes the risk composition. Our findings are twofold:

Our quantitative risk analysis reveals a clear stratification of risk that is strongly correlated with the applications’ business models. Categories whose core functions involve processing sensitive personal data to facilitate real-world transactions or services—such as Travel & Transportation (Avg. PRS: 23.13) and E-commerce & Retail (23.45)—consistently exhibit the highest risk scores. In stark contrast, categories that primarily focus on delivering information or entertainment, such as Content & Entertainment (7.97), show significantly lower risk. The median PRS scores corroborate this finding, indicating that the high risk in the former categories is systemic and not merely the effect of a few outliers. Furthermore, this stratification is evident not only in the average risk scores, but also in the composition of risk within each category. As illustrated in Figure 5, in high-risk categories like Travel & Transportation, a staggering 56.73% of all apps fall into the High or Severe risk tiers. For E-commerce & Retail, this proportion is even higher at 58.66%. In contrast, for the low-risk Content & Entertainment category, only 13.67% of the MiniApps fall into these top levels. This stark contrast empirically validates that a MiniApp’s business model is a strong predictor of both the magnitude and the composition of its privacy risk.

Our large-scale study reveals a troubling landscape where basic transparency is widely neglected and data minimization principles are undermined through a spectrum of issues, from clear-cut violations to subtle, coercive designs. By quantifying these issues with our risk model, we demonstrate that the associated risks are not uniform. Instead, they are disproportionately concentrated in applications whose core business models rely on processing sensitive data for real-world transactions and services. These findings underscore the urgent need for scalable, automated auditing tools like MiniEval to effectively govern the vast and opaque MiniApp ecosystem.

6. Discussion

Our large-scale analysis using the MiniEval framework reveals systemic deficiencies in the privacy practices of the MiniApp ecosystem. These deficiencies manifest across a spectrum of issues, from the high prevalence of Omitted Disclosures to the subtle coercion of Indirect Violations. Our quantitative assessment further demonstrates that these privacy risks are not uniformly distributed; instead, they are disproportionately concentrated

Table 7: Key Risk Metrics Across Major MiniApp Categories, Sorted by Average PRS. The "High/Severe Risk %" column now represents the proportion of apps classified into the top two original tiers (Elevated/High and Severe/Extreme).

Category	Count	Avg	Median	High/Severe%
Real Estate	108	23.72	23.10	61.11 %
Medical & Health	319	23.59	23.10	60.81 %
E-commerce & Retail	1974	23.45	20.90	58.66 %
Travel & Transportation	379	23.13	20.90	56.73 %
Enterprise & Professional Services	1711	22.86	20.90	57.57 %
Government & Public Services	165	21.67	20.90	61.82 %
Local & Life Services	1061	21.28	18.70	52.03 %
Finance & Payment	277	20.75	19.10	52.35 %
Agriculture	110	19.79	16.50	48.18 %
Sports	81	18.28	16.20	44.44 %
Tools	306	15.51	12.10	39.54 %
Social Networking	495	14.27	12.10	30.71 %
Education	1072	12.80	8.80	29.29 %
Information & News	630	10.51	1.10	26.83 %
Content& Entertainment	3789	7.97	1.10	13.67 %

in data-driven application categories, such as Travel & Transportation and E-commerce & Retail. This empirically establishes that an application’s business model is a strong predictor of its privacy risk posture.

These findings have significant implications for the governance of MiniApp platforms. The systemic lack of transparency and the widespread use of coercive patterns indicate that existing governance mechanisms, which rely heavily on platform policies and developer self-regulation, are inadequate. This highlights the necessity for scalable, automated auditing tools like MiniEval that advance beyond rudimentary violation detection to offer a nuanced, quantitative risk assessment. Such tools can empower regulators to prioritize enforcement, enable platforms to conduct more effective oversight, and allow users to make genuinely informed decisions. Our work thus provides a methodology to translate abstract privacy principles into measurable, verifiable metrics, offering a new foundation for governing the complex MiniApp ecosystem.

We acknowledge several limitations in our current work, which also outline avenues for future research. First, our static analysis engine struggles to fully

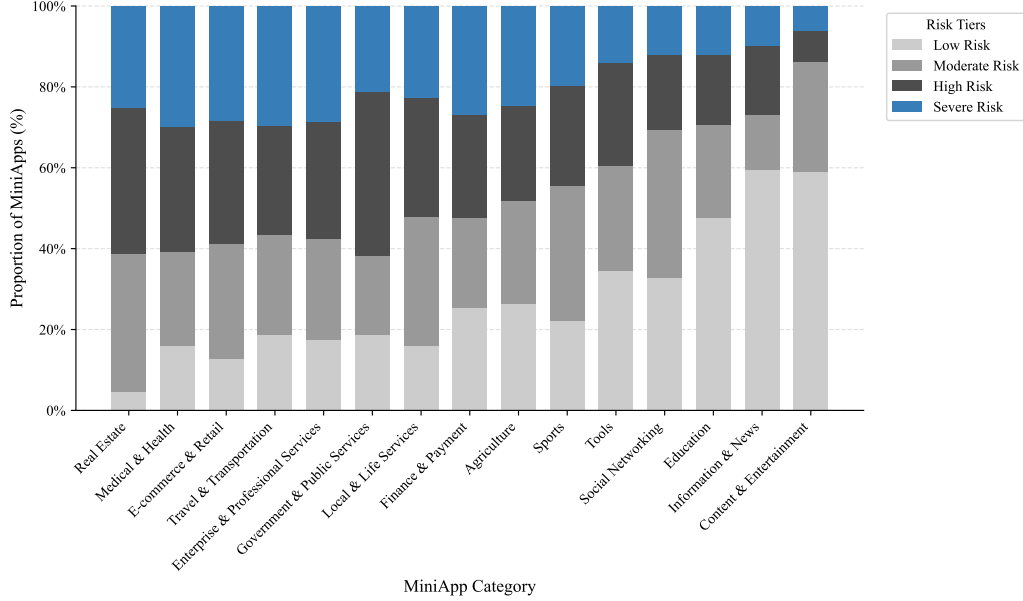


Figure 5: Risk composition varies significantly across MiniApp categories. The plot shows the proportion of apps in each of the four risk tiers, with categories sorted by their average PRS.

comprehend the architectural complexities of modern front-end ecosystems. These ecosystems, which include frameworks like `Uni-app`, state management libraries such as `Vuex`, and underlying bundlers like `Webpack`, introduce layers of abstraction that obscure the information essential for precise violation detection. Consequently, our analyzer cannot always reconstruct the complete context needed to accurately assess the relationship between data collection and its functional purpose, leading to detection inaccuracies.

These limitations define a clear trajectory for future work. A primary direction is to enhance our static analysis engine to robustly analyze applications built with third-party frameworks. This involves developing techniques to de-obfuscate control flows in bundled code and modeling framework-specific abstractions. Furthermore, building on our current use of the LLM for Scenario Inconsistency detection, we plan to deepen this multimodal analysis. By more effectively leveraging the visual context from UI screenshots, we aim to resolve semantic ambiguities that are intractable through code analysis alone. To refine the risk model, we will investigate machine learning techniques to learn sensitivity weights and risk parameters directly from data,

reducing the reliance on heuristics. Finally, a key priority is to extend the MiniEval framework to other major MiniApp platforms, such as Alipay and TikTok, to validate its adaptability and provide a more comprehensive assessment of the broader privacy landscape.

7. Related Work

7.1. WeChat MiniApp Security

Research on WeChat MiniApp security has progressively deepened in scope and sophistication. Early efforts focused on establishing foundational knowledge and detection frameworks. For instance, Wang Tao et al. [19] conducted the first empirical study on 83 WeChat MiniApp errors and developed a detection tool named WeDetector. Building on this, Han et al. [20] proposed a comprehensive trinity threat model for MiniApps, while Yang et al. [18] targeted a specific attack vector by developing CmrfsScanner to detect Cross MiniApp Request Forgery (CMRF) vulnerabilities. Subsequently, research delved deeper into the API and permission layers, uncovering more subtle implementation flaws. For example, Li et al. [21] identified critical flaws such as sensitive information leakage through improper file management, while Che et al. [22] systematically studied "API-scope misalignment," a novel vulnerability arising from incorrect mappings between APIs and their permission scopes. In addition to these technical vulnerabilities, researchers have also focused on abusive behaviors during the permission request process. For instance, Wang et al. developed MiniChecker [23] to address coercive and harassing permission requests. By constructing a universal function call graph, MiniChecker automatically identifies five types of abusive request patterns, such as homepage, repeating, and looping pop-ups. Another critical line of research has focused on tracking the flow of sensitive data to detect privacy leaks directly. Tools like TaintMini [24], for instance, utilized static taint analysis to detect sensitive data flows. Similarly, MiniTracker [16] conducted a large-scale study on 150k WeChat MiniApps to reveal common privacy leakage patterns. Furthermore, other studies have begun to examine the consistency between an app's actual data practices and its stated privacy policy. In this area, Yin Wang et al. [15] found that as many as 89.4% of applications violated their own privacy policies. [8].

7.2. Software Risk Quantification and Assessment

Enck et al. [25] proposed a system to identify whether an application uses potentially dangerous permission combinations. To this end, they manually defined a set of permission combinations representing risk. If an application uses these combinations through static analysis, it is labeled as a malicious application. H Hao et al. [26] constructed a standard permission vector model by statistically analyzing the permission request distribution of benign applications in the same category. They then assessed the risk value of Android applications by computing the Euclidean distance between the application’s permission vector and the standard model. HX Son et al. [27, 28] proposed a hybrid analysis-based method. They extracted application permission requests and data collection behaviors through static analysis to generate an application signature. By comparing the application signature of the target application with the category signature and combining semantic similarity, they measured the degree of deviation in permission requests. They also adjusted weights using dynamic behavior analysis to generate more accurate risk assessment results. Merlo and Georgiu [29] introduced RiskInDroid, a tool that effectively assesses the risk of Android applications. RiskInDroid defined feature vectors based on four types of application permissions and applied supervised learning by using specific classifiers. The PRADroid framework [30] evaluated the likelihood of risk by using an ensemble classifier composed of five classifiers, such as decision trees. It extracts and encodes 24 dangerous permission features from applications.

8. Conclusion

We introduced MiniEval, a framework for automated detection of violations and quantitative privacy risk assessment in MiniApps. By integrating hybrid analysis with Large-Language-Model-based reasoning, MiniEval identifies both explicit policy violations and subtle coercive behaviors, translating these findings into a holistic Privacy Risk Score. Our empirical evaluation on a large-scale dataset of real-world MiniApps demonstrates the framework’s effectiveness and yields two key findings. First, we provide quantitative evidence that privacy flaws in the MiniApp ecosystem are systemic, ranging from explicit policy violations to subtle forms of Indirect Violation. Second, our analysis establishes a strong correlation between an application’s functional category and its privacy risk, with data-driven applications consistently posing a higher risk to users. These findings highlight the limitations of current

governance practices and underscore the necessity of scalable, automated auditing mechanisms. Future work will focus on enhancing the adaptability of our risk model and extending the MiniEval framework to other MiniApp platforms. This research provides a foundational methodology for fostering a more transparent and accountable mobile ecosystem.

9. Example Appendix Section

Appendix text.

Example citation, See [?].

References

- [1] European Commission, General data protection regulation, https://commission.europa.eu/law/law-topic/data-protection_en, accessed: 8 sep 2025 (2022).
- [2] State of California Department of Justice, California consumer privacy act, <https://oag.ca.gov/privacy/ccpa>, accessed: 8 sep 2025 (2022).
- [3] Personal Information Protection Committee, Act on the protection of personal information, <https://www.ppc.go.jp/>, accessed: 8 sep 2025 (2022).
- [4] the Government of Canada, Consumer Privacy Protection Act, <https://ised-isde.canada.ca/site/innovation-better-canada/en/consumer-privacy-> accessed: 8 sep 2025 (2022).
- [5] Tencent WeChat Team, Wechat privacy policy, https://www.wechat.com/en/privacy_policy.html, accessed: 8 sep 2025 (2023).
- [6] X. Zhang, Y. Wang, X. Zhang, Z. Huang, L. Zhang, M. Yang, Understanding privacy over-collection in wechat sub-app ecosystem (2023). arXiv:2306.08391, doi:10.48550/arXiv.2306.08391. URL <https://arxiv.org/abs/2306.08391>
- [7] S. Wang, Y. Li, K. Wang, Y. Liu, H. Li, Y. Liu, H. Wang, Miniscope: Automated ui exploration and privacy inconsistency detection of miniapps via two-phase iterative hybrid analysis, ACM Trans. Softw.

- Eng. Methodol. 34 (6) (Jul. 2025). doi:10.1145/3709351.
URL <https://doi.org/10.1145/3709351>
- [8] S. Wang, Y. Zhao, K. Wang, H. Wang, On the usage-scenario-based data minimization in mini programs, in: Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Superapps, SaTS '23, Association for Computing Machinery, New York, NY, USA, 2023, p. 29–32. doi:10.1145/3605762.3624435.
URL <https://doi.org/10.1145/3605762.3624435>
 - [9] W3C, Miniapp subpackaging, accessed: 8 sep 2025 (2023).
URL <https://www.w3.org/TR/mini-app-white-paper/#subpackaging>
 - [10] Online documentation, accessed: 8 sep 2025 (2025).
URL <https://docs.google.com/spreadsheets/d/1BzMkHFyPR5UCWj-eZzuC6IREqJioEhCL/>
 - [11] Tencent WeChat Team, Wxml syntax reference / conditional rendering, <https://developers.weixin.qq.com/miniprogram/dev/reference/wxml/conditional.html>, accessed: 8 sep 2025.
 - [12] S. Li, Z. Yang, Y. Yang, D. Liu, M. Yang, Identifying cross-user privacy leakage in mobile mini-apps at a large scale, IEEE Trans. Inf. Forensics Secur. 19 (2024) 3135–3147. doi:10.1109/TIFS.2024.3356197.
 - [13] unveiler, unveiler, [Computer software] Accessed: 8 sep 2025.
URL <https://github.com/0day404/unveilr>
 - [14] Frida, Frida, <https://github.com/frida/frida>, [Computer software] Accessed: 8 sep 2025.
 - [15] Y. Wang, M. Fan, J. Liu, J. Tao, W. Jin, H. Wang, Q. Xiong, T. Liu, Do as you say: Consistency detection of data practice in program code and privacy policy in mini-app, IEEE Trans. Softw. Eng. 50 (12) (2024) 3225–3248. doi:10.1109/TSE.2024.3479288.
 - [16] W. Li, B. Yang, H. Ye, L. Xiang, Q. Tao, X. Wang, C. Zhou, Minitracker: Large-scale sensitive information tracking in mini apps, IEEE Trans. Dependable Secure Comput. 21 (4) (2024) 2099–2114. doi:10.1109/TDSC.2023.3299945.

- [17] State Administration for Market Regulation and Standardization Administration of China , Data security technology—Security requirements for processing of sensitive personal information (GB/T 45574-2025), National Standard of the People’s Republic of China, accessed: 8 sep 2025 (Apr. 2025).
- [18] Y. Yang, Y. Zhang, Z. Lin, Cross miniapp request forgery: Root causes, attacks, and vulnerability detection, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS ’22, Association for Computing Machinery, New York, NY, USA, 2022, p. 3079–3092. doi:10.1145/3548606.3560597.
URL <https://doi.org/10.1145/3548606.3560597>
- [19] T. Wang, Q. Xu, X. Chang, W. Dou, J. Zhu, J. Xie, Y. Deng, J. Yang, J. Yang, J. Wei, T. Huang, Characterizing and detecting bugs in wechat mini-programs, in: Proceedings of the 44th International Conference on Software Engineering, ICSE ’22, Association for Computing Machinery, New York, NY, USA, 2022, p. 363–375. doi:10.1145/3510003.3510114.
URL <https://doi.org/10.1145/3510003.3510114>
- [20] Y. Han, X. Ji, Z. Wang, J. Zhang, Systematic analysis of security and vulnerabilities in miniapps, in: Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Superapps, SaTS ’23, Association for Computing Machinery, New York, NY, USA, 2023, p. 1–9. doi:10.1145/3605762.3624432.
URL <https://doi.org/10.1145/3605762.3624432>
- [21] J. Li, Y. Zhang, W. Zhou, S. Zhang, Exploring permission control flaws in mini-apps, in: 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2024, pp. 1935–1941. doi:10.1109/TrustCom63139.2024.00268.
- [22] J. Che, C. Guo, N. Dong, J. Pei, L. Fan, X. Mi, X. Xie, X. Luo, Z. Liu, R. Cheng, Uncovering api-scope misalignment in the app-in-app ecosystem, Proceedings of the ACM on Software Engineering 2 (ISSTA) (2025) 1933–1954. doi:<https://doi.org/10.1145/3728962>.
- [23] Y. Wang, M. Fan, H. Zhou, H. Wang, W. Jin, J. Li, W. Chen, S. Li, Y. Zhang, D. Han, et al., Minichecker: Detecting data privacy risk of abusive permission request behavior in mini-programs,

- in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1667–1679. doi:<https://doi.org/10.1145/3691620.3695534>.
- [24] C. Wang, R. Ko, Y. Zhang, Y. Yang, Z. Lin, Taintmini: Detecting flow of sensitive data in mini-programs with static taint analysis, in: Proceedings of the 45th International Conference on Software Engineering, ICSE '23, IEEE Press, Melbourne, Victoria, Australia, 2023, p. 932–944. doi:[10.1109/ICSE48619.2023.00086](https://doi.org/10.1109/ICSE48619.2023.00086).
URL <https://doi.org/10.1109/ICSE48619.2023.00086>
 - [25] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, A. N. Sheth, Taintdroid: an information-flow tracking system for realtime privacy monitoring on smartphones, *ACM Trans. Comput. Syst.* 32 (2) (2014) 1–29. doi:<https://doi.org/10.1145/2619091>.
 - [26] An effective approach to measuring and assessing the risk of android application, in: 2015 International Symposium on Theoretical Aspects of Software Engineering, 2015, pp. 31–38. doi:[10.1109/TASE.2015.16](https://doi.org/10.1109/TASE.2015.16).
 - [27] H. X. Son, B. Carminati, E. Ferrari, A risk assessment mechanism for android apps, in: 2021 IEEE International Conference on Smart Internet of Things (SmartIoT), 2021, pp. 237–244. doi:[10.1109/SmartIoT52359.2021.00044](https://doi.org/10.1109/SmartIoT52359.2021.00044).
 - [28] H. X. Son, B. Carminati, E. Ferrari, A risk estimation mechanism for android apps based on hybrid analysis, *Data Sci. Eng.* 7 (3) (2022) 242–252. doi:<https://doi.org/10.1007/s41019-022-00189-1>.
 - [29] A. Merlo, G. C. Georgiu, Riskindroid: Machine learning-based risk analysis on android, in: ICT Systems Security and Privacy Protection: 32nd IFIP TC 11 International Conference, SEC 2017, Rome, Italy, May 29-31, 2017, Proceedings 32, Springer, 2017, pp. 538–552. doi:https://doi.org/10.1007/978-3-319-58469-0_36.
 - [30] Y. Yang, X. Du, Z. Yang, Pradroid: Privacy risk assessment for android applications, in: 2021 IEEE 5th International Conference on Cryptography, Security and Privacy (CSP), 2021, pp. 90–95. doi:[10.1109/CSP51677.2021.9357608](https://doi.org/10.1109/CSP51677.2021.9357608).